P0476r1
Bit-casting object representations

Published Proposal, 11 November 2016

This version:
http://wg21.link/P0476r1

Author:
JF Bastien (Apple)

Audience:
LEWG, LWG, CWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0476r1.bs

Implementation:
github.com/jfbastien/bit_cast/

Abstract

Obtaining equivalent object representations The Right Way™.

Table of Contents

This paper is a revision of [P0476r0], addressing LEWG comments from the 2016 Issaquah meeting. See §4 Revision History for details.

§ 1. Background

Low-level code often seeks to interpret objects of one type as another: keep the same bits, but obtain an object of a different type. Doing so correctly is error-prone: using `reinterpret_cast` or `union` runs afoul of type-aliasing rules yet these are the intuitive solutions developers mistakenly turn to.

Attuned developers use `aligned_storage` with `memcpy`, avoiding alignment pitfalls and allowing them to bit-cast non-default-constructible types.

This proposal uses appropriate concepts to prevent misuse. As the sample implementation demonstrates we could as well use `static_assert` or template SFINAE, but the timing of this library feature will likely coincide with concept's standardization.

Furthermore, it is currently impossible to implement a `constexpr` bit-cast function, as `memcpy` itself isn't `constexpr`. Marking the proposed function as `constexpr` doesn't require or prevent `memcpy` from becoming `constexpr`. This leaves implementations free to use their own internal solution (e.g. LLVM has [a `bitcast` opcode]).

We should standardize this oft-used idiom, and avoid the pitfalls once and for all.

§ 2. Proposed Wording

Below, substitute the � character with a number the editor finds appropriate for the sub-section.

§ 2.1. Synopsis

Under 20.2 Header `<utility>` synopsis [**utility**]:

```
namespace std {
  // ...

  // 20.2.� bit-casting:
  template<typename To, typename From>
  requires
    sizeof(To) == sizeof(From) &&
    is_trivially_copyable_v<To> &&
    is_trivially_copyable_v<From>
  constexpr To bit_cast(const From& from) noexcept;

  // ...
}
```

§ 2.2. Details

Under 20.2.� Bit-casting [**utility.bitcast**]:

```
template<typename To, typename From>
requires
  sizeof(To) == sizeof(From) &&
  is_trivially_copyable_v<To> &&
  is_trivially_copyable_v<From>
constexpr To bit_cast(const From& from) noexcept;
```

1. Requires: `sizeof(To) == sizeof(From)`, `is_trivially_copyable_v<To>` is `true`, `is_trivially_copyable_v<From>` is `true`.
2. Returns: an object of type `To` whose *object representation* is equal to the object representation of `From`. If multiple *object representations* could represent the *value representation* of `From`, then it is unspecified which `To` value is returned. If no *value representation* corresponds to `To`'s *object representation* then the returned value is unspecified.

§ 2.3. Feature testing

The `__cpp_lib_bit_cast` feature test macro should be added.

§ 3. Appendix

The Standard's **[basic.types]** section explicitly blesses `memcpy`:

> For any trivially copyable type `T`, if two pointers to `T` point to distinct `T` objects `obj1` and `obj2`, where neither `obj1` nor `obj2` is a base-class subobject, if the *underlying bytes* (1.7) making up `obj1` are copied into `obj2`, `obj2` shall subsequently hold the same value as `obj1`.
>
> [*Example:*
>
> ```
> T* t1p;
> T* t2p;
> // provided that t2p points to an initialized object ...
> std::memcpy(t1p, t2p, sizeof(T));
> // at this point, every subobject of trivially copyable type in *t1p contains
> // the same value as the corresponding subobject in *t2p
> ```
>
> — *end example*]

Whereas section [class.union] says:

> In a union, at most one of the non-static data members can be active at any time, that is, the value of at most one of the non-static data members can be stored in a union at any time.

§ 4. Revision History

§ 4.1. r0 ➡ r1

The paper was reviewed by LEWG at the 2016 Issaquah meeting:

- Remove the standard layout requirement—trivially copyable suffices for the `memcpy` requirement.
- We discussed removing `constexpr`, but there was no consent either way. There was some suggestion that it'll be hard for implementers, but there's also some desire (by the same implementers) to have those features available in order to support things like `constexpr` instances of `std::variant`.
- The pointer-forbidding logic was removed. It was initially there to help developers when a better tool is available, but it's easily worked around (e.g. with a `struct` containing a pointer). Note that this doesn't prevent `constexpr` versions of `bit_cast`: the implementation is allowed to error out on `bit_cast` of pointer.
- Some discussion about concepts-usage, but it seems like mostly an LWG issue and we're reasonably sure that concepts will land before this or in a compatible vehicle.

Straw polls:

- Do we want to see [P0476r0] again? unanimous consent.
- `bit_cast` should allow pointer types in `To` and `From`. **SF F N A SA** 4 5 4 2 1
- `bit_cast` should be `constexpr`? **SF F N A SA** 4 3 7 2 3

§ 5. Acknowledgement

Thanks to Saam Barati, Jeffrey Yasskin, and Sam Benzaquen for their early review and suggested improvements.

§ References

§ Informative References

[P0476r0]
JF Bastien. Bit-casting object representations. 16 October 2016. URL: https://wg21.link/p0476r0